

Lab 5: Implementing an Independent Microservice

System: BadNet - Badminton Social Network Chosen Microservice: **Match Service** (Core Feature)

Introduction

This lab focuses on the practical implementation of one of the core services identified in Lab 4: the **Match Service**. This service is completely independent, owns its own data (Challenges & Matches), and exposes a RESTful API, strictly adhering to microservices principles.

Unlike the monolithic Laravel application, this microservice:

- Runs independently on its own port
- Owns its dedicated database tables
- Exposes well-defined API contracts for other services
- Can be deployed and scaled separately

Objectives

1. Set up a standalone Laravel application dedicated to Challenge & Match management.
2. Implement the Match Service logic and persistence.
3. Expose the defined Service Contract (REST API).
4. Test the service in complete isolation.

Technology & Tool Installation

- **Laravel 11** (PHP Framework)
- **MySQL** (Dedicated database)
- **Postman / curl** (API Testing)

Activity Practice 1: Project Setup

1. Create Independent Service Directory

Bash

```
mkdir match-service
```

```
cd match-service
```

```
composer create-project laravel/laravel .
```

2. Configure Environment

Update .env:

env

APP_NAME="BadNet Match Service"

APP_URL=http://localhost:8001

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=badnet_match

DB_USERNAME=root

DB_PASSWORD=

Run on different port to ensure isolation

SERVER_PORT=8001

3. Create Dedicated Database

SQL

```
CREATE DATABASE badnet_match CHARACTER SET utf8mb4 COLLATE  
utf8mb4_unicode_ci;
```

Activity Practice 2: Data Modeling & Service Implementation

Domain Model

File: app/Models/Challenge.php

PHP

<?php

```
namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
```

```
class Challenge extends Model
```

```
{
```

```
    use HasFactory;
```

```
    protected $fillable = [
```

```
        'challenger_id',
```

```
        'opponent_id',
```

```
        'match_date',
```

```
        'location',
```

```
        'status',
```

```
        'score_challenger',
```

```
        'score_opponent',
```

```
        'elo_change_challenger',
```

```
        'elo_change_opponent',
```

```
        'note'
```

```
    ];
```

```
    public function challenger()
```

```
    {
```

```
        return $this->belongsTo(User::class, 'challenger_id');
```

```
    }
```

```
    public function opponent()
```

```
    {
```

```
        return $this->belongsTo(User::class, 'opponent_id');
    }
}
```

Repository (Data Access)

File: app/Repositories/ChallengeRepository.php

PHP

<?php

```
namespace App\Repositories;
```

```
use App\Models\Challenge;
```

```
class ChallengeRepository
```

```
{
    public function create(array $data)
    {
        return Challenge::create($data);
    }
}
```

```
public function findById(int $id)
{
    return Challenge::with(['challenger', 'opponent'])->find($id);
}
```

```
public function getAll()
{
    return Challenge::with(['challenger', 'opponent'])->latest()->paginate(15);
}
```

```
}  
}
```

Service Layer

File: app/Services/ChallengeService.php

PHP

```
<?php
```

```
namespace App\Services;
```

```
use App\Repositories\ChallengeRepository;
```

```
class ChallengeService
```

```
{
```

```
    protected $repository;
```

```
    public function __construct(ChallengeRepository $repository)
```

```
    {
```

```
        $this->repository = $repository;
```

```
    }
```

```
    public function createChallenge(array $data)
```

```
    {
```

```
        if ($data['challenger_id'] == $data['opponent_id']) {
```

```
            throw new \Exception('Cannot challenge yourself');
```

```
        }
```

```
        return $this->repository->create($data);
```

```
}
```

```
public function getChallenge(int $id)
{
    $challenge = $this->repository->findById($id);
    if (!$challenge) {
        throw new \Exception('Challenge not found');
    }
    return $challenge;
}
}
```

Presentation Layer (Controller)

File: app/Http/Controllers/ChallengeController.php

PHP

```
<?php
```

```
namespace App\Http\Controllers;

use App\Services\ChallengeService;
use Illuminate\Http\Request;

class ChallengeController extends Controller
{
    protected $service;

    public function __construct(ChallengeService $service)
    {
```

```

    $this->service = $service;
}

public function store(Request $request)
{
    try {
        $challenge = $this->service->createChallenge($request->all());
        return response()->json([
            'success' => true,
            'message' => 'Challenge created successfully',
            'data' => $challenge
        ], 201);
    } catch (\Exception $e) {
        return response()->json([
            'success' => false,
            'message' => $e->getMessage()
        ], 400);
    }
}

```

```

public function show($id)
{
    try {
        $challenge = $this->service->getChallenge($id);
        return response()->json([
            'success' => true,
            'data' => $challenge
        ], 200);
    } catch (\Exception $e) {
        return response()->json([
            'success' => false,
            'message' => $e->getMessage()
        ], 400);
    }
}

```

```

    });
    } catch (\Exception $e) {
        return response()->json([
            'success' => false,
            'message' => $e->getMessage()
        ], 404);
    }
}
}
}

```

Route (routes/api.php):

PHP

```
Route::apiResource('challenges', ChallengeController::class);
```

How to Run (Isolation Test)

Bash

```
php artisan serve --port=8001
```

Test Endpoints (using Postman or curl):

- POST <http://localhost:8001/api/challenges>
- GET <http://localhost:8001/api/challenges/1>

Activity Practice 3: Isolation Testing & Verification

Goal: Verify that the Match Service operates correctly as a standalone, independent microservice.

Step-by-Step Instructions

1. Start the Match Service

Bash

```
cd match-service
```

```
php artisan serve --port=8001
```

The service should run on <http://127.0.0.1:8001>

2. Test Create Challenge (POST)

- **Endpoint:** POST `http://127.0.0.1:8001/api/challenges`

- **Body:**

JSON

```
{  
  "challenger_id": 1,  
  "opponent_id": 3,  
  "match_date": "2026-05-25 15:00:00",  
  "location": "Quan 7 Badminton Arena",  
  "note": "Friendly match"  
}
```

- **Expected Result:** HTTP 201 Created with challenge data.

3. Test Get Challenge Details (GET)

- **Endpoint:** GET `http://127.0.0.1:8001/api/challenges/1`

- **Expected Result:** HTTP 200 OK returning full challenge information with related challenger and opponent.

4. Test Error Handling

- **Endpoint:** GET `http://127.0.0.1:8001/api/challenges/999` (non-existent ID)
- **Expected Result:** HTTP 404 Not Found with appropriate error message.

5. Isolation Verification

- Stop the main Laravel monolith (php artisan serve on port 8000).
- Confirm the Match Service still works normally on port **8001**.
- This proves the service is truly independent.

Summary of Deliverables

- Project structure of the Match Service
- Code of the Challenge Model, Repository, Service, and Controller
- Screenshots of successful API testing (POST & GET)_sProof that the service runs independently on port 8001